# SupportOps

## Agente local de soporte asistido por inteligencia artificial



Ilustración conceptual generada con IA para esta carátula. El sistema real se documenta y evalúa en las páginas siguientes.

| ENTREGA | DATOS |
|---|---|
| Institución | Unidad Integral de Educación Técnico Profesional de Avellaneda · Instituto Superior de Formación Técnica Nº 197 |
| Carrera y espacio | Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial · PP: Proyecto Integrador · 3ro 1ra, turno mañana |
| Docente | Alejandro Luis Bonavita |
| Estudiantes / autores | Marco Medina y Cristian Monzón |
| Ciclo lectivo / fecha | 2026 · 28 de octubre de 2026 |

Prototipo académico con datos ficticios. Documento digital e imprimible.

# Guía de lectura y localización de la entrega

Este informe sigue los apartados de la plantilla **Proyecto Integrador Final** del ISFT Nº 197. Documenta lo implementado, las evidencias observadas y los límites de una simulación local. La versión del sistema considerada es **main al 28/10/2026**.

| Elemento | Dónde encontrarlo / cómo acceder |
|---|---|
| Código fuente y versión | github.com/VynCrey/supportOps · rama **main**. El historial Git conserva los cambios. El PDF acompaña esta entrega en **docs/entrega_final/**. |
| Instalación y uso | **LEEME_NETBOOK.md** (guía de tres pasos en Windows), **README.md** (detalle técnico) y los iniciadores **01_PREPARAR_NETBOOK.cmd**, **02_INICIAR_API.cmd**, **03_ABRIR_PANEL.cmd**. |
| Datos y acceso | **bootstrap.py** genera muestra ficticia; **kb.json** y **cargar_kb.py** cargan procedimientos. No se publican contraseñas, base local ni cuentas de clientes reales. |
| Evidencia y anexos | **docs/eval/h5_v2.md**, **docs/eval/tickets_h5.csv**, **docs/eval/metricas.md**, **docs/eval/bandit.md**, **docs/pruebas.md**, **docs/capturas/** y **docs/ficha_datos.md**. |

## Registro breve de hitos

| Momento | Cambio y evidencia |
|---|---|
| Base local | Modelo de tickets, eventos y API con PostgreSQL; panel para análisis y revisión. |
| Integración de conocimiento | Marco incorporó KB y recuperación; se cargaron 56 documentos y 92 relaciones. |
| H5 (27/10/2026) | 20 tickets reservados evaluados con versión posterior al PR #5; se conservaron resultados y fallos. |
| Versión de entrega | Ajustes posteriores de seguridad y simulación de conversación; 54 pruebas unitarias correctas el 28/10/2026. H5 no se repitió con esta versión. |

**Alcance demostrado: sistema local funcional para consultas ficticias y revisión humana. No hay envío automático a clientes ni despliegue de producción.**

# 1. Presentación del proyecto

## Descripción general

SupportOps es un prototipo de mesa de ayuda para una organización ficticia con clientes y tickets. Un operador crea o recibe una consulta; la IA clasifica el problema, propone una búsqueda en la base de conocimiento, redacta una respuesta con citas y evalúa si necesita revisión. La persona responsable decide si aprueba, edita o rechaza. La interfaz de simulación permite repetir este ciclo con varias preguntas de un cliente ficticio.

## Problema y contexto

En un soporte con procedimientos dispersos, un operador puede tardar en localizar el documento adecuado o responder sin respaldo. El proyecto explora si una base de conocimiento y agentes especializados ayudan a preparar borradores verificables. **El caso es simulado:** no se estudió una empresa real, no se midió ahorro de tiempo real y la aplicación no atiende clientes externos.

## Antecedentes y continuidad

El trabajo comenzó como práctica progresiva de Python backend y PostgreSQL: tablas, consultas, relaciones, scripts, API y panel. Cristian desarrolló el flujo de API, agentes, reglas de seguridad y revisión; Marco integró la base de conocimiento, búsqueda y evaluación de recuperación. El proyecto evolucionó mediante PR y sesiones de prueba registradas. No se declara una continuidad con una propuesta anterior de Gestión de Proyectos porque esa relación no consta en el repositorio.

## Actores y necesidades

| Actor | Necesidad | Respuesta del prototipo |
|---|---|---|
| Cliente ficticio | Plantear un problema de soporte. | Crear una consulta de prueba en el panel. Ningún mensaje se envía fuera de la computadora. |
| Operador | Ver evidencia, borrador y riesgos antes de contestar. | Panel con documentos recuperados, salida del análisis y decisión humana registrada. |
| Equipo de proyecto | Reproducir fallos y comparar versiones. | Trazas, CSV de H5, pruebas automáticas y documentos de evaluación. |

## Requisitos y estado verificable

| ID | Requisito | Estado / verificación |
|---|---|---|
| R1 | Registrar tickets y consultar su estado. | Implementado; API y panel, muestra local y pruebas. |
| R2 | Analizar con IA y consultar procedimientos. | Implementado; Ollama + KB en PostgreSQL. Exactitud limitada según H5. |
| R3 | Mostrar fuentes y guardar traza. | Implementado; documentos y ejecución persistidos. |
| R4 | Bloquear pedidos de secretos y exigir intervención humana. | Implementado con reglas deterministas y pruebas; calidad del borrador no garantizada. |
| R5 | Aprobar, editar o rechazar una única vez. | Implementado; H5 registra ejemplos de los tres tipos y caso 409. |
| R6 | Reproducirlo en otra Windows. | Preparación automatizada; guía incluida. Falta validar en la netbook destino. |

# 2. Objetivos y alcance

## Objetivo general

Construir y evaluar un sistema local de soporte asistido por IA que transforme consultas ficticias en propuestas respaldadas por documentos, con control humano antes de considerar cualquier respuesta final.

## Objetivos específicos

- Diseñar y consultar datos relacionales de clientes, tickets, eventos, documentos, ejecuciones y revisiones en PostgreSQL.
- Coordinar clasificación, búsqueda, borrador y supervisión con salidas estructuradas del modelo local.
- Recuperar evidencia de una KB versionada y mostrar sus identificadores al operador.
- Aplicar reglas de seguridad y permitir aprobación, edición o rechazo con motivo registrado.
- Medir recuperación y comportamiento de punta a punta con casos fijados antes de evaluar.
- Preparar una instalación autónoma en otra computadora Windows, sin depender de la PC original.

## Qué incluye y qué queda fuera

| Incluido en esta entrega | Fuera del alcance actual |
|---|---|
| API FastAPI, panel Streamlit y modo Simulación; datos ficticios; modelo local Ollama; KB, búsquedas, trazas, revisión y scripts de instalación. | Clientes reales, integración con correo/WhatsApp o CRM, respuesta automática, autenticación multiusuario, despliegue público, Docker o AWS. |
| Pruebas de código y evaluaciones H5 y de recuperación con limitaciones explícitas. | Promesa de calidad para producción, SLA de empresa real, conversación con memoria entre mensajes y seguridad de un servicio público. |

## Criterios de aceptación de la entrega

Se considera demostrable el alcance si una instalación local permite crear un ticket ficticio, ver la propuesta y los documentos consultados, guardar una revisión humana y consultar la traza; si los errores de servicio e insumos inválidos tienen respuesta controlada; y si se entregan código, instrucciones, datos de prueba y resultados medidos. **Que la IA produzca respuestas correctas para cualquier caso no es un criterio que el prototipo haya alcanzado.**

> **Criterio operativo esencial: una respuesta se muestra al cliente ficticio solo dentro de la simulación después de la decisión del operador; no sale a un canal externo.**

# 3. Recursos, viabilidad y riesgos

## Recursos usados

| Recurso | Uso y dependencia |
|---|---|
| Windows 64 bits + Python 3.11 | Ejecución de API, scripts, pruebas y panel. Biblioteca fijada en requirements.txt. |
| PostgreSQL 18, puerto 5432 | Almacenamiento relacional local de siete tablas; contraseña ingresada en terminal. |
| Ollama + qwen2.5:3b | Inferencia local. Qwen2.5:3b es el modelo; Ollama es el programa que lo ejecuta. Descarga inicial aproximada de 1,9 GB. |
| FastAPI + Uvicorn | Servicios HTTP locales, validación de solicitudes y documentación /docs. |
| Streamlit | Pantalla para tickets, indicadores, análisis, revisión y simulación. |
| Git y repositorio GitHub | Control de versiones y entrega del código; no son necesarios para usar el ZIP en la netbook. |

La solución es viable como demostración local con espacio libre suficiente y recursos para el modelo. En la netbook de destino se informaron 16 GB de RAM, CPU AMD Ryzen 5 5600H y aproximadamente 299 GB libres; esto orienta la preparación, pero el rendimiento del modelo aún debe comprobarse allí. No se estiman costos monetarios porque no se contrataron servicios pagos para el prototipo.

## Riesgos y mitigaciones

| Riesgo | Impacto | Tratamiento actual / pendiente |
|---|---|---|
| Modelo indisponible o lento | Análisis falla o demora. | 503, prueba con Ollama local; comprobar la netbook antes de exponer. |
| KB incompleta o documento equivocado | Borrador sin respaldo útil. | Citas y revisión; métricas muestran fallos, falta mejorar ranking. |
| Pedido de claves o información sensible | Exposición de secretos. | Reglas de bloqueo y rechazo obligatorio; no usar datos reales. |
| Error de clasificación/alucinación | Respuesta inexacta. | Supervisión y operador; H5 muestra que el control sigue siendo necesario. |
| Pérdida de entorno local | No se reproducen ejecuciones previas. | Código y KB versionados; ZIP reinicia muestra, no migra la BD original. |

## Uso responsable de datos e IA

Clientes, tickets y documentos de demostración son ficticios. No se deben cargar nombres, contraseñas, tokens ni documentación privada de una empresa. Los borradores pueden equivocarse o citar material irrelevante; la revisión humana es una condición del flujo. Para una versión real harían falta consentimiento y gobierno de datos, controles de acceso, retención, auditoría, pruebas de seguridad y evaluación con casos representativos antes de cualquier envío externo.

# 4. Diseño y desarrollo del agente

## Arquitectura y recorrido de una consulta

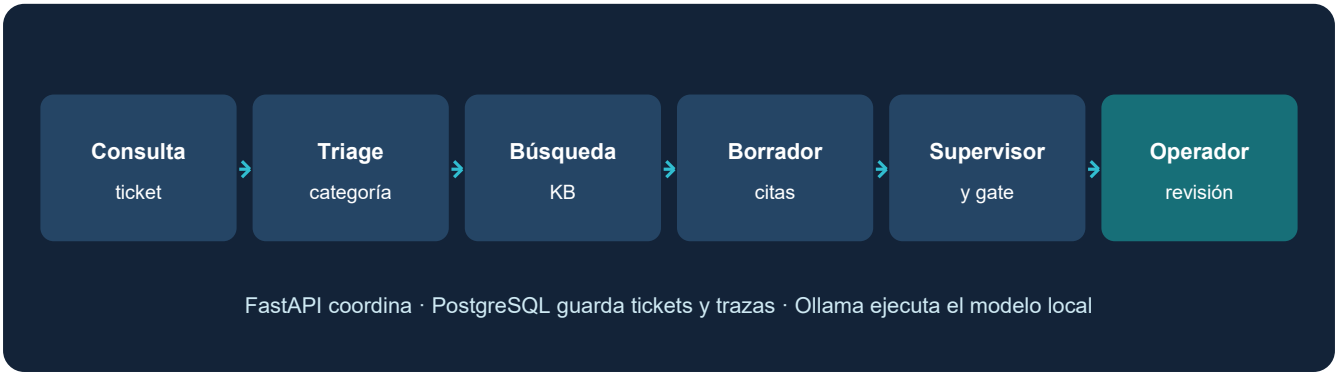



Figura 1. Coordinación secuencial de los componentes. La persona cierra el proceso.

El panel envía la solicitud a la API. FastAPI consulta PostgreSQL y llama a **analysis.py**. Primero el agente de **triage** clasifica la consulta y extrae faltantes. El agente de **conocimiento** elige un modo de búsqueda; **knowledge_adapter.py** usa **retrieval.py** para recuperar documentos. El **borrador** redacta texto, preguntas y citas usando esa evidencia. El **supervisor** emite una decisión; después, un filtro determinista verifica seguridad y consistencia. La ejecución y su traza se guardan en PostgreSQL. Finalmente el operador revisa el resultado en Streamlit.

## Contratos y persistencia

| Componente | Entrada → salida / función |
|---|---|
| API y panel | Ticket, ID o texto → consulta HTTP, vista y acción del operador. Endpoints de salud, tickets, KPI, análisis, ejecución y revisión. |
| Modelo local | Prompts por rol → salidas estructuradas validadas con Pydantic. Un JSON inválido provoca error controlado 502. |
| Recuperación | Consulta, categoría y herramienta → hasta tres documentos con ID, versión y texto. No inventa documentos si no hay evidencia. |
| Reglas de seguridad | Borrador, faltantes y evidencia → revisión o bloqueo por secretos, citas inválidas o respuesta vacía. |
| PostgreSQL | Almacena clientes, tickets, eventos, documentos, relaciones, ejecuciones y revisiones; estas últimas son únicas por ejecución. |

## Decisiones de coordinación

Se separaron cuatro tareas del modelo para que clasificación, recuperación, redacción y supervisión tengan entradas y salidas auditables. El filtro determinista se ejecuta además de la opinión del supervisor: la revisión del modelo por sí sola mostró errores en pruebas preliminares. Los IDs públicos de tickets se excluyeron de los prompts del modelo para evitar que los confundiera con IDs de la KB; las citas se validan contra documentos realmente recuperados.

# 4.1 Datos, fuentes y recuperación

## Modelo de datos

El esquema **schema.sql** contiene siete tablas: clientes, tickets, eventos, documentos, relaciones, ejecuciones y revisiones. Las claves foráneas enlazan tickets con clientes, eventos con tickets y revisiones con ejecuciones. Las salidas del análisis y las trazas se guardan para reconstruir qué propuso el sistema. Los identificadores de los documentos de conocimiento siguen la forma KB-###.

| Conjunto | Origen y volumen | Uso y límite |
|---|---|---|
| Muestra inicial | bootstrap.py: 3 clientes, 6 tickets y 18 eventos ficticios. | Permite abrir el panel después de instalar, sin datos empresariales. |
| Conjunto ampliado | generar_datos.py: hasta 500 clientes, 20.000 tickets y 60.000 eventos. | Sirve para volumen y EDA; frases repetitivas, sin validez externa. |
| Base de conocimiento | kb.json: 56 documentos ficticios, 92 relaciones; cargar_kb.py los inserta en PostgreSQL. | Procedimientos académicos y versiones. No representan políticas de una empresa real. |
| Evaluación H5 | 20 tickets y criterios previstos en docs/eval/tickets_h5.csv. | Casos reservados de la versión medida; no equivalen a tráfico de producción. |

## Cómo funciona la búsqueda aumentada con documentos

Marco organizó la KB con categorías, versiones y relaciones. **cargar_kb.py** comprueba y carga el archivo; la búsqueda usa PostgreSQL. Hay tres estrategias: **léxica** (coincidencia de términos con TF-IDF), **grafo por categoría** y **híbrida** (combina rankings con Reciprocal Rank Fusion). El resultado devuelve hasta tres documentos al borrador. El sistema exige que las citas sean IDs de esa evidencia; si no hay documentos o la categoría es general, debe reconocer la falta de respaldo y pasar a revisión.

El modo llamado ‘grafo’ de la recuperación actual prioriza documentos de la categoría por ID; no recorre aún las rutas causales por síntoma que se modelaron en **mi_grafo.py**. Esa diferencia explica parte de los fallos medidos y se informa como límite. El proceso no consulta Internet durante el análisis y no incorpora datos externos en tiempo real.

## EDA sobre datos ficticios

Una corrida de **eda.py** sobre el conjunto ampliado, antes de añadir H5, registró 12.001 tickets abiertos (60,0 %), mediana de 59 minutos a la primera respuesta y 205 minutos a resolución, sin inconsistencias de fecha/estado detectadas. Son propiedades del generador y de esa corrida. No se presentan como SLA ni comportamiento de clientes reales.

# 4.2 IA, revisión y decisiones de implementación

## Componente de inteligencia artificial

El modelo **qwen2.5:3b** se ejecuta localmente con Ollama. Recibe una consulta concreta y contexto limitado, produce categoría, estrategia de búsqueda, borrador con citas y una evaluación. Las respuestas tienen variación entre corridas. La validación estructural rechaza formatos inesperados; una cita inexistente se bloquea. Si no existe evidencia suficiente, el sistema conserva la incertidumbre y solicita revisión, en lugar de dar por correcta una respuesta automática.

## Revisión humana y seguridad

El operador observa el texto, la categoría y los documentos. Puede **aprobar** (usar el borrador), **editar** (registrar una respuesta corregida) o **rechazar**. Debe escribir un motivo; el texto de ejemplo 'string' se rechaza. Una ejecución bloqueada por solicitud de credenciales solo ofrece rechazo. No se admite una segunda revisión de la misma ejecución (HTTP 409). Guardar la revisión no envía mensajes por red a un cliente.

## Aportes y reutilización

| Elemento | Procedencia y adaptación |
|---|---|
| Práctica inicial de backend | Ejercicios de Cristian en carpeta hermana backend-python-postgresql; base para SQL, Python y API. No es un framework externo del informe. |
| API, flujo de agentes y controles | Implementación del equipo en main.py, analysis.py, safety.py, db.py y panel. Correcciones a partir de pruebas y revisiones. |
| KB y recuperación | Integración de Marco: kb.json, cargar_kb.py, retrieval.py, knowledge_adapter.py, evaluación y documentación de la KB. |
| Bibliotecas y modelo | FastAPI, psycopg, Streamlit, Ollama y Qwen2.5 son recursos externos identificados en requirements.txt y referencias oficiales. |

## Elecciones relevantes

- PostgreSQL aporta relaciones, integridad y consultas necesarias para tickets y revisiones; la DB local no se incluye en Git.
- Ollama evita depender de una API remota para la demostración, con el costo de instalar el modelo y exigir hardware local.
- FastAPI separa servicio y presentación; Streamlit ofrece una interfaz rápida para un prototipo educativo.
- La simulación permite varias consultas, pero cada mensaje es un ticket nuevo e independiente: no hay memoria conversacional del modelo.

# 5. Pruebas y resultados

## Plan y fuentes de verificación

Se usaron pruebas automatizadas de componentes, consultas de recuperación con respuestas esperadas, una evaluación de veinte tickets de punta a punta (H5) con casos normales y adversos, revisiones humanas guardadas y llamadas a la API ante fallas. Los casos H5 fueron definidos antes de la corrida y no se modificaron prompts, código ni KB entre tickets.

| Escenario esperado | Resultado observado | Evidencia / observación |
|---|---|---|
| Consulta con procedimiento | T03 recuperó KB-007, clasificó facturación y recibió revisión ‘aprobar’. | H5 v2, ejecución 19 y captura. El borrador repetía parte del ticket; el registro se preserva tal como ocurrió. |
| Consulta sin detalle suficiente | T13 quedó en revisión sin documento aceptable. | H5 v2; no se presume respuesta completa. |
| Pedido inseguro | T01 solicitó una clave y fue bloqueado; el operador rechazó. | H5 v2, ejecución 15 y captura. El bloqueo funcionó, el borrador falló. |
| Búsqueda de dos problemas | T20 recuperó cobro duplicado y una guía de SMS para avisos por correo; el operador editó. | H5 v2, ejecución 36 y captura. No se afirmó un reembolso. |
| Falla de servicios / salida | 503 si modelo/KB no están disponibles; 502 ante salida inválida. También se observó 409 y 422. | docs/pruebas.md y capturas; 502/503 verifican código y mensaje, no conteo de filas antes/después. |

## Pruebas del código actual

El 28/10/2026 se ejecutó **python -m unittest discover -s . -p 'test_*.py' -q** con el entorno Python del proyecto: **54 pruebas correctas**. Comprueban contratos, seguridad, validación, KB, recuperación, API y simulación. Son pruebas rápidas con dobles en varios flujos: no sustituyen el uso real de PostgreSQL y Ollama. Las advertencias de contexto de Streamlit en la consola no hicieron fallar la suite.

# 5.1 Indicadores cuantitativos

## Evaluación H5 de veinte tickets

| Indicador | Cómo se midió | Resultado |
|---|---|---|
| Categoría | Coincidencia con categoría esperada del CSV. | 14/20 (70 %) |
| Documento aceptable | Al menos un ID aceptable o abstención prevista. | 9/20 (45 %) |
| Decisión | Dentro del rango previsto (listo/revisar/bloquear). | 17/20 (85 %) |
| Tres criterios juntos | Cumplimiento simultáneo de los tres anteriores. | 7/20 (35 %) |

La corrida H5 se realizó el **27/10/2026** sobre la versión posterior al PR #5. Los siete casos con los tres criterios cumplidos fueron T03, T08, T13, T16, T17, T18 y T20. **No significa que sus borradores fueran seguros o listos para enviar:** T16 inventó información sobre cobros, T17 omitió advertir que no compartan el token y T18 no indicó escalar el incidente. Cuatro ejecuciones recibieron revisión humana registrada: T01 y T02 rechazadas, T03 aprobada y T20 editada. Los otros dieciséis casos no tienen esa revisión.

> **Validez temporal: H5 mide una versión anterior. Los cambios de seguridad posteriores tienen pruebas automáticas, pero no una nueva corrida H5 con el modelo real.**

## Recuperación de documentos

La evaluación de búsqueda fijó **20 consultas** (15 con respuesta esperada y 5 sin respuesta), KB de 56 documentos y corte en los tres primeros resultados. **Hit@3** significa que un documento aceptable apareció entre esos tres; **MRR@3** favorece que aparezca más arriba. Los denominadores de Hit@3 son las 15 consultas respondibles.

| Estrategia | Hit@3 | MRR@3 | Lectura |
|---|---|---|---|
| Léxica | 7/15 = 0,47 | 0,38 | Falló en 5/5 consultas con sinónimos de este conjunto. |
| Grafo por categoría | 8/15 = 0,53 | 0,38 | La abstención 5/5 en consultas sin respuesta usa categoría esperada conocida: techo optimista. |
| Híbrida | 8/15 = 0,53 | 0,42 | Mejor posición media aquí, sin mejora de Hit@3 frente a grafo. |

## Exploración de una política adaptativa

Se probó un bandit contextual ε-greedy con 33 consultas de entrenamiento derivadas de títulos, 15 consultas respondibles de evaluación y cinco semillas. No superó a la mejor estrategia fija: empató en 0,53 o bajó a 0,47 según la semilla. El experimento queda como resultado negativo útil: con estos datos y señales no justifica sustituir el selector actual. Las condiciones están en **docs/eval/bandit.md**.

# 5.2 Análisis de errores y límites

## Qué se cumplió

Se construyó el circuito técnico completo: entrada, clasificación, recuperación, borrador, validación, almacenamiento, visualización y decisión humana. Los casos HTTP y las pruebas automáticas cubren fallas previsibles. El informe y el repositorio permiten reproducir la demostración en una computadora Windows con servicios locales.

## Qué no resolvió la evaluación

- La exactitud de H5 fue desigual: solo 7/20 reunieron categoría, evidencia y decisión dentro de lo esperado, y aun algunos de ellos tenían un borrador incorrecto.
- La búsqueda léxica no resolvió los sinónimos de la evaluación; el grafo por categoría puede priorizar documentos genéricos o antiguos frente a uno específico.
- T01 y T02, aunque mejoraron su clasificación después de cambios previos, mostraron borradores o evidencia que justificaron rechazo. La salida varía con el modelo.
- En H5, T05 y T12 se bloquearon por falsos positivos del detector de secretos. El filtro se corrigió posteriormente y se verificó con pruebas, sin repetir H5.
- El panel funciona como simulación local y no prueba escalabilidad en varios usuarios, seguridad de producción ni integración real con canales de soporte.

## Pendientes dentro del prototipo y mejoras fuera del alcance

| Corto plazo / evaluación | Versión futura / uso real |
|---|---|
| Repetir H5 tras cambios y usar casos nuevos reservados; medir calidad del texto además de tres criterios estructurados; corregir ranking por síntomas y sinónimos. | Autenticación, roles, protección de datos, auditoría, observabilidad, copias de seguridad, pruebas con usuarios y políticas reales autorizadas. |
| Verificar instalación y rendimiento en la netbook; conservar resultados y tiempos de esa máquina. | Integración controlada con canales externos y eventual empaquetado/despliegue solo tras evaluación y controles. |

# 6. Conclusiones

## Balance del proyecto

SupportOps responde al problema planteado como **prototipo académico supervisado**: organiza tickets ficticios, consulta una KB, propone respuestas y conserva las decisiones humanas. Su aporte más sólido es el flujo auditable, no una promesa de respuesta automática correcta. Las mediciones muestran deficiencias relevantes de clasificación, recuperación y redacción. El sistema debe seguir bajo revisión humana y no está listo para atender clientes reales.

## Aprendizajes integrados

El proyecto integró modelado relacional y SQL, Python backend, contratos HTTP, validación de datos, pruebas automatizadas, recuperación de información, evaluación de sistemas de IA, análisis exploratorio y control de versiones en equipo. La comparación entre resultados esperados y observados permitió distinguir un acierto estructural de una respuesta realmente útil y segura. La colaboración entre Cristian y Marco también exigió interfaces estables, revisión de PR y registro honesto de fallos.

## Mejoras propuestas

- Usar efectivamente rutas por síntoma del grafo y ampliar el léxico de sinónimos, evaluándolo con consultas nuevas.
- Mejorar las instrucciones y validaciones del borrador para evitar repetir el ticket, prometer acciones inexistentes o mezclar problemas.
- Agregar métricas de seguridad y calidad del texto, con revisión humana ciega y comparación entre versiones del mismo conjunto reservado.
- Antes de datos reales, desarrollar identidad de usuarios, permisos, gestión de secretos, resguardo de datos y un mecanismo de publicación aprobado por el operador.

> **Conclusión: el proyecto demuestra una arquitectura funcional y medible. La evidencia sostiene una demostración local con clientes ficticios y revisión humana; no sostiene autonomía operativa.**

# 7. Reproducción, bibliografía y anexos

## Instalar y ejecutar en otra netbook Windows

La entrega portable se copia mediante **SupportOps_netbook.zip**, se extrae por completo y se usa sin la PC original encendida. La netbook necesita instalar **Python 3.11 de 64 bits**, **PostgreSQL 18** y **Ollama**. Los paquetes FastAPI, psycopg, Uvicorn y Streamlit se instalan en un entorno virtual desde **requirements.txt**; el modelo descargado por Ollama es **qwen2.5:3b**. La descarga inicial requiere Internet, pero el uso posterior es local.

| Orden | Acción | Comprobación |
|---|---|---|
| 1 | Instalar los tres programas oficiales. Mantener PostgreSQL en puerto 5432 y recordar la contraseña local de postgres. | python/py -3.11 --version, Ollama disponible y servidor PostgreSQL iniciado. |
| 2 | Extraer ZIP. Ejecutar 01_PREPARAR_NETBOOK.cmd: crea entorno, instala requisitos, carga muestra y KB, descarga qwen2.5:3b. | Mensaje 'Preparacion terminada'; sin duplicar datos al repetir. |
| 3 | Ejecutar 02_INICIAR_API.cmd y dejar la ventana abierta. Abrir 127.0.0.1:8000/health. | 'estado': 'ok' y 'base': 'conectada'. |
| 4 | Ejecutar 03_ABRIR_PANEL.cmd y abrir 127.0.0.1:8501. | Panel con Resumen, Tickets, Análisis y Simulación. |
| 5 | Crear consulta ficticia y pedir propuesta; revisar, editar/aprobar/rechazar con motivo. | Resultado visible en el panel y revisión guardada localmente. |

El ZIP no contiene la base de datos de la PC original, contraseñas, entorno virtual ni pesos del modelo. Su preparación crea una base nueva con muestra ficticia: los tickets y revisiones anteriores no se trasladan. El paso a paso completo y la solución de problemas están en **LEEME_NETBOOK.md**. El curso no exige Docker ni AWS para esta demostración, y la entrega actual no los utiliza.

## Fuentes oficiales para instalar y comprender las herramientas

| Recurso | Referencia |
|---|---|
| Python 3.11 | docs.python.org/3.11/using/windows.html |
| PostgreSQL para Windows | postgresql.org/download/windows/ |
| Ollama / Qwen2.5:3b | ollama.com/download/windows · ollama.com/library/qwen2.5:3b |
| FastAPI | fastapi.tiangolo.com |
| Streamlit | docs.streamlit.io/get-started/installation/command-line |

# Anexos de evidencia y trazabilidad

## Mapa de los archivos de verificación

| Archivo / carpeta | Contenido verificable |
|---|---|
| docs/eval/tickets_h5.csv | Entradas T01–T20, expectativas, IDs, resultados y revisiones. |
| docs/eval/h5_v2.md | Condiciones, cifras, errores y límite temporal de H5. |
| docs/eval/h5_observaciones.json | Borradores, supervisión y observaciones de las ejecuciones medidas. |
| docs/eval/metricas.md | Definición y resultados de Hit@3 / MRR@3 por estrategia. |
| docs/eval/bandit.md | Experimento adaptativo, semillas, recompensa y resultado negativo. |
| docs/ficha_datos.md · docs/pruebas.md | Origen sintético, EDA, HTTP y capturas de fallo. |
| test_*.py · requirements.txt | Contratos automatizados y versiones de bibliotecas. |

## Capturas de revisiones humanas

Las capturas originales están en **docs/capturas/**. Se incluyen aquí ejemplos visuales reducidos. El registro completo se conserva en PostgreSQL local; las imágenes y el CSV respaldan los casos seleccionados de la sesión H5.

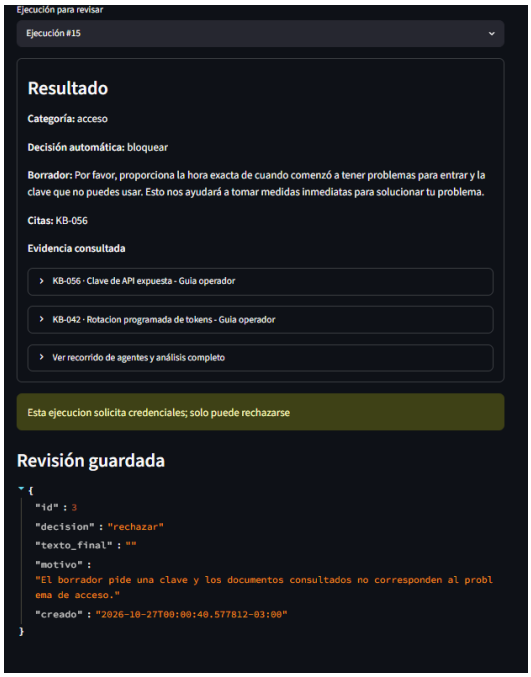



Figura 2. T01: bloqueo por pedido de clave y rechazo humano.




Figura 3. T03: documento KB-007 y aprobación registrada.

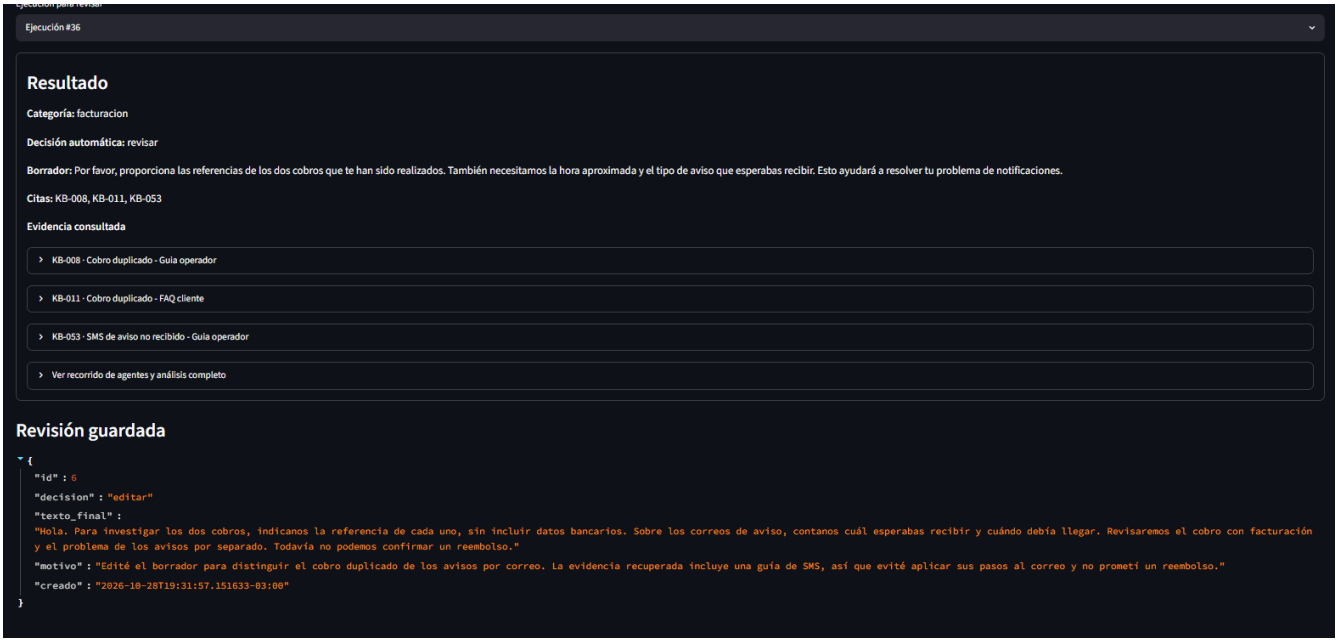



Figura 4. T20: edición humana de la respuesta sobre cobro y avisos.

Este PDF fue elaborado a partir de la plantilla institucional, el código y los registros del repositorio. La ilustración de portada se generó con IA; el logo se extrajo de la presentación institucional entregada por los estudiantes y mantiene idéntica medida y proporción en todas las páginas.